certora

Security Assessment

Alluvial finance – Sms lite

May 2025

Prepared for Alluvial finance

certora

Table of Contents

certora

Project Summary

Project Scope

Project Name	Repository (link)	Commit Hash	Platform
sms-lite	GitHub repository	d3ce772	EVM
tvs	GitHub repository	2c4095f	EVM

Project Overview

This document describes the manual code review of the Alluvial SMS-lite and TVS implementations.

The work was a 15-day effort undertaken from **25/04/2025** to **09/05/2025**

The following contract list is included in our scope:

- sms-lite/src/OrchestratorV1.sol
- sms-lite/src/OrchestratorUpgradeableBase.sol
- sms-lite/src/libraries/LibUnstructuredStorage.sol
- sms-lite/src/libraries/LibBytes.sol
- sms-lite/src/libraries/LibUint256.sol
- sms-lite/src/OrchestratorFactory.sol
- sms-lite/src/state/orchestratorUpgradeable/ConfigManagerAddress.sol
- sms-lite/src/state/orchestratorUpgradeable/BeneficiaryAddress.sol
- sms-lite/src/state/orchestratorUpgradeable/TVSBeaconAddress.sol
- sms-lite/src/state/orchestratorFactory/InstanceID.sol
- sms-lite/src/state/orchestratorFactory/OrchestratorImplementationAddress.sol
- sms-lite/src/components/ConfigManager.sol
- sms-lite/src/components/state/configManager/AllocationConfig.sol
- sms-lite/src/components/state/configManager/MinSweepAmount.sol
- sms-lite/src/OrchestratorBase.sol

- sms-lite/src/batchContracts/BatchSweep.sol
- sms-lite/src/batchContracts/BatchTVSTransfer.sol
- sms-lite/src/batchContracts/BatchPectra.sol
- sms-lite/src/batchContracts/state/DepositContractAddress.sol
- sms-lite/src/batchContracts/BatchDeposit.sol
- sms-lite/src/batchContracts/BatchSetTVSBeneficiary.sol
- sms-lite/src/batchContracts/BatchUpgradeableTVSCreator.sol

- tvs/src/TVSUpgradeable/ImmutableBeaconFactory.sol
- tvs/src/TVSUpgradeable/state/proxy/Beacon.sol
- tvs/src/TVSUpgradeable/TVSUpgradeable.sol
- tvs/src/TVSUpgradeable/ImmutableBeacon.sol
- tvs/src/TVSUpgradeable/proxies/TVSBeaconProxy.sol
- tvs/src/state/Beneficiary.sol
- tvs/src/components/TVS.sol
- tvs/src/components/BaseSecurity.sol
- tvs/src/TVSNonUpgradeable/TVSClone.sol
- tvs/src/TVSNonUpgradeable/TVSImmutable.sol
- tvs/src/TVSNonUpgradeable/TVSImmutableBase.sol
- tvs/src/TVSNonUpgradeable/TVSFlexibleImmutable.sol

Alluvial's SMS-lite and TVS offer an innovative way of abstracting native staking on Ethereum on a large scale.

Users can use it as a single platform to manage validators operated by multiple service providers, and can efficiently exchange groups of validators without requiring them to exit their positions. TVS supports some of the newest staking features of the Pectra upgrade for increased flexibility.

Certora helped with focused scrutiny and testing to secure the implementation and the novel integrations.

The team performed a manual audit of all the Solidity smart contracts. During the manual audit, the Certora team discovered bugs in the Solidity smart contracts code, as listed on the following pages.

Risks outside of the audit scope

On top of the issues further reported as bugs, the Certora team identified systemic risks related to the parts of the overall design that are outside of the scope of the audit:

- Assumptions on TVS transfers outside of the audit scope: as TVS transfers move ownership of assets, we understand that these will most often be part of a sale of entities that trust (and are trusted by) Alluvial but don't necessarily trust each other. Our understanding is that these sales will be secured through an on-chain Escrow contract that will hold ownership of the transferred TVS for the time necessary to perform off-chain checks. We assume that off-chain checks will be exhaustive and will cover the many ways the previous owner could have tampered with the TVS.
  To cite a few non-exhaustive examples of what should be covered:
  - Assets locked in the consensus and execution layer
  - All the storage slots used by TVS that could have been altered through a malicious upgrade:
    - `BEACON_SLOT` (as there is no guarantee that `ImmutableBeacon` is used), and the consequent implementation
    - `INITIALIZABLE_STORAGE` (as there is no guarantee that the TVS hasn't been rigged to be re-initializable)
    - `BENEFICIARY_SLOT` (as when this is altered, the seller can sweep value out of the TVS after the sale price has been fixed by the off-chain agent)
    - `OwnableStorageLocation` (for the integrity of the exchange: if the transfer to the escrow has been faked i.e. via forged events or custom getters, the seller can make changes after the sale price has been fixed)

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	2	2	2
Low	7	6	5
Informational	14	11	9
Total	23	19	16

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Medium Severity Issues

M-01 Double-spend due to using msg.value in a loop

Severity: **Medium**	Impact: **Low**	Likelihood: **High**
Files: BatchPectra.sol	Status: Fixed	

Description: Consolidation and CL Withdrawal operations batched across TVS instances by the `BatchPectra` orchestrator functions are performed within a for loop. The problem lies with using `msg.value` within a loop (lines 61 and 97). Because the amount sent with the call will be spent in the first iteration, it won't cover further iterations, causing them to deviate from the intended behavior by using the contract balance to cover fees or failing when the contract balance is insufficient.

JavaScript

```
File: BatchPectra.sol
059:         for (uint256 i = 0; i < _consolidationSelection.length; i++) {
060:             ConsolidationSelection calldata selection = _consolidationSelection[i];
061:             ITVS(...).consolidate{ value: msg.value }(
062:                 selection._consolidationRequests, _maxFeePerRequest, _excessFeeRecipient
063:             );
064:         }
---
093:         for (uint256 i = 0; i < _validatorWithdrawSelections.length; i++) {
---
097:             ITVS(...).withdraw{ value: msg.value }(
098:                 _validatorWithdrawSelecdaions[i]._publicKeys,
099:                 _validatorWithdrawSelections[i]._amount,
100:                 _maxFeePerRequest,
101:                 _excessFeeRecipient
102:             );
```

```
103:         }
```

Exploit Scenario: Any call with requests for CL withdrawal or consolidations across more than one TVS will deplete the Orchestrator balance or fail.

Recommendations: Consider accepting a per-TVS value parameter, and adding a further requirement that these parameters should sum up to `msg.value`

Customer's response: Fixed with 044d09b & 49bc630

Fix Review: Fixes confirmed.

M-02 Orchestrators cannot upgrade received TVS instances

Severity: **Medium**	Impact: **Low**	Likelihood: **High**
Files: OrchestratorBase.sol	Status: Fixed	

Description: As per the specification of TVS transfers, the Orchestrator owner should be allowed to upgrade TVS instances.

The Orchestrator contract, however, offers no entry point to allow this upgrade, so the orchestrator owner cannot change the beacon of upgradeable TVS instances.

Alternatively, a TVS upgrade could also be achieved by changing the implementation pointed by the existing beacon; however, this option is certainly not feasible when an Orchestrator receives

a TVS via `transfer`, because in this situation, the beacon is set to `ImmutableBeacon` where the implementation is fixed.

Exploit Scenario: When a `TVSUpgradeable` instance is transferred, the receiving Orchestrator cannot call `setBeacon`, and it must further transfer it to a different address that can make this call.

Recommendations: Consider adding a `setBeacon` entry point to Orchestrator contracts.

Customer's response: Fixed with [6663cac]

Fix Review: Fix confirmed

Low Severity Issues

L-01 `SWEEPER_ROLE` isn't verified correctly

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: BatchSweep.sol, OrchestratoBase.sol	Status: Fixed	

Description: Currently, there are 2 functions that execute sweeps in the protocol, `sweepToBeneficiary` & `sweepAndTransferTVSs`. Neither functions require `SWEEPER_ROLE` in order to execute. In the case of `sweepAndTransferTVSs` it only requires the `VALIDATOR_SET_TRANSFER_ROLE`, but it also sweeps.

Exploit Scenario: Anyone can call `sweepToBeneficiary` permissionlessly & `sweepAndTransferTVSs` can be called by a sender with only the `VALIDATOR_SET_TRANSFER_ROLE` and not the `SWEEPER_ROLE`.

Recommendations: Override `_sweepToBeneficiary` and allow only a caller with the `SWEEPER_ROLE` to be able to use it. Add an extra check in `sweepAndTransferTVSs`, that checks if the caller has the `SWEEPER_ROLE`.

Customer's response: Fixed with 3256ab3

Fix Review: Fix confirmed

L-02 transferOrchestrator misses check for to != msg.sender

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: OrchestratorBase.sol	Status: Fixed	

Description: The OrchestratorBase contract has a safety net in place that prevents the contract owner from renouncing their `DEFAULT_ADMIN` role. This check, however, can be bypassed with even greater damage if the contract owner transfers the orchestrator to themselves i.e. by mistake or in an attempt to fix some of their groups.

JavaScript
File: OrchestratorBase.sol
```
289:     function _transferOrchestrator(address _to) internal {
290:         if (_to == address(0)) revert InvalidAddress();
291:         _configureOwner(_to);
292:         _revokeRoles(msg.sender, _getOwnerRoles());
293:     }
```

When this `transferOrchestrator(self)` call is made, setting up new roles for _to does nothing because the caller already has those roles, but the next call to `_revokeRoles` strips the sender out of all their admin roles.

Exploit Scenario: When the Orchestrator owner self-transfers the instance, this ends up with no owner.

Recommendations: Consider adding the check `to != msg.sender`

Customer's response: Fixed with [6fca925]

Fix Review: Fix confirmed. Revoking before granting is an elegant solution.

L-03 Orchestrator can't be upgraded while paused

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: OrchestratorBase.sol	Status: Fixed	

Description: The `OrchestratorBase.transferOrchestrator` function is decorated with the `isNotPaused` modifier:

```
JavaScript
File: OrchestratorBase.sol
283:     /// @inheritdoc IOrchestrator
284:     function transferOrchestrator(address to) external isNotPaused
onlyRole(DEFAULT_ADMIN_ROLE) {
```

Exploit Scenario: In case an emergency upgrade for the Orchestrator is required, the typical flow is:

- Pausing the orchestrator

- Performing an upgrade
- Unpausing the orchestrator

This flow would not be possible with the current isNotPaused check.

Recommendations: Consider removing the `isNotPaused` modifier from the `transferOrchestrator` function.

Customer's response: Fixed with 9b17f27

Fix Review: Fix confirmed

L-04 Missing key length check allows for emitting broken events

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: TVS.sol	Status: Fixed	

Description: The `consolidate` and `withdraw` functions accept validator public keys in the form of a `bytes` Solidity parameter which is accepted at variable lengths. These functions don't validate that the length is of the correct size, which is 48 bytes.

Exploit Scenario: While this missing check is not a problem for the `withdraw` function, whose execution flow would revert in the withdraw EL contract, for the `consolidate` function, the source and target public keys are concatenated, so one can pass 0 bytes for the source key and 96 bytes for the target key, and have a transaction that does not revert. This execution flow would emit a `ConsolidationRequested` event that is malformed, and which could throw off off-chain software that is unlikely to be equipped with proper logic to handle the situation.

Recommendations: Consider adding 48-bytes constraint to the public keys accepted by the `consolidate` function at least, and ideally also `withdraw`.

Customer's response: Fixed with 238f261

Fix Review: Fix confirmed

L-05 Non-compliance with ERC-1967 BeaconUpgraded event

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: TVSUpgradeable.sol	Status: Fixed	

Description: The `TVSUpgradeable` contract, when its beacon is updated, emits a BeaconUpdated event as follows:

```
JavaScript
File: TVSUpgradeable.sol
80:     function setBeaconUnchecked(address newBeacon) external onlyOwner {
81:         address oldBeacon = Beacon.get();
82:         Beacon.set(newBeacon);
83:         emit BeaconUpdated(oldBeacon, newBeacon);
84:     }
```

This event does not follow the ERC-1967 specification, which states:

```
Unset
Storage slot [...] (obtained as bytes32(uint256(keccak256('eip1967.proxy.beacon')) - 1)).

[...] Changes to this slot SHOULD be notified by the event:

> event BeaconUpgraded(address indexed beacon);
```

Exploit Scenario: TVS beacon upgrades will emit an event that is likely not recognized by off-chain monitoring tools designed around the ERC-1967 standard.

Recommendations: Change the event at L83 to match that of the ERC-1967 standard, or, if the current format is preferred, consider emitting both events.

Customer's response: Fixed with b56ee17

Fix Review: Fix confirmed

L-06 Old roles are preserved after Orchestrator is transferred

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: OrchestratorBase.sol	Status: Not Fixed	

Description: The function `Orchestrator.transferOrchestrator` is used to transfer ownership of an Orchestrator contract to a different address.

As per our understanding, this different address may be another address of the same entity (i.e. a different multisig controlled by the same company), but also an address controlled by a different entity.

In the latter case, the transfer is only partial because `transferOrchestrator` only reassign owner roles - other roles like those related to `keeper` and, more importantly, `staker`, are not reassigned:

```
JavaScript
File: OrchestratorBase.sol
283:     /// @inheritdoc IOrchestrator
284:     function transferOrchestrator(address to) external isNotPaused
onlyRole(DEFAULT_ADMIN_ROLE) {
285:         _transferOrchestrator(to);
286:         emit OrchestratorTransferred(msg.sender, to);
```

```
287:     }
288:
289:     function _transferOrchestrator(address _to) internal {
290:         if (_to == address(0)) revert InvalidAddress();
291:         _configureOwner(_to);
292:         _revokeRoles(msg.sender, _getOwnerRoles());
293:     }
```

Exploit Scenario: Entity Alice Corp transfers their orchestrator to Bob Inc. After this is done, any roles configured in the orchestrator to addresses other than the primary owner are kept, and Alice Corp can keep performing withdrawal operations.

Recommendations: On top of strong off-chain checks on contract integrity, using `AccessControlEnumerable` should allow for resetting all roles at transfer time.

If there is no strong requirement for allowing Orchestrator transfers among entities, limiting inter-entity transfers to TVS only would greatly reduce the attack surface and consequently risk of valuable asset transfers.

Customer's response: Acknowledged - the reported behavior is by design: orchestrator is not designed to be moved between organizations/entities. To eliminate further ambiguity, we renamed the `transferOrchestrator` function to `changeOwner` in commit [75703b3]

L-07 Consolidation queue time can be weaponized to steal value after TVS transfer

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: TVS.sol	Status: Not Fixed	

Description: The TVS integration with the EIP-7251 consolidation contract allows TVS owners to trigger validator consolidations.

As EIP-7251 treats consolidation requests with a queuing mechanism that processes at most 2 consolidations per block, a meaningful amount of time (up to hours) may pass between the submission and execution of a consolidation request.

This behavior applies to EL-initiated withdrawals too, however representing a much more limited threat to the protocol.

Exploit Scenario: If a TVS transfer happens during this period, it is possible that:

- The TVS owner initiates a consolidation request targeting a public key they control
- The TVS is escrowed and analyzed for pricing
- The price is set and the TVS is transferred
- The consolidation request gets executed, and value is extracted from the TVS after the sale

Recommendations: we recommend adding a check that prevents transfers from happening while there is a pending consolidation request. This check could be implemented on-chain (via an oracle or by reverse engineering congestion from the consolidation fee, or, maybe more easily, off-chain while the TVS is escrowed. Alternatively, a minimum escrow delay, also empirically based on the fee formula, could be enforced before off-chain checks kick in.

Customer's response: Acknowledged - the TVS transfer is going to initially be possible only between trusted parties. We will have off-chain checks & agreements in place to prevent attacks like this; we will build on-chain checks when we start moving TVSs into LC protocol and other liquid staking protocols.

Informational Severity Issues

I-01 Misleading natspec for EIP-7002 withdraw amounts

Description: For EL withdrawals, we have callers passing a `uint64` amount, and these values are pass-through for Orchestrator and TVS.

However, the documentation of these values seems misleading:

```
JavaScript
File: ITVS.sol
176:      * @param amount The respective amounts to withdraw from
each of the validators. Zero amount means full exit
---
File: BatchPectra.sol
31:         uint64[] _amount; // The amount to withdraw from a
validator
```

The amount is in `gwei` (because the type is `uint64` and not `uint256`), and this is not reflected in the natspec, suggesting a possible misinterpretation / increased likelihood of errors in the integrating logic.
We are also not sure that the `Zero amount means full exit` assumption is accurate, as we did not find supporting evidence in the EIP-7002 spec.

Recommendation: We recommend modifying the natspec to clearly specify that the amount is in `gwei`, and checking that zero amounts effectively initiate full exits.

Customer's response: Fixed with 459dbee

Fix Review: Fix confirmed.

I-02 TVS._assertOwner is reimplementing OwnableUpgradeable._checkOwner

Description: The function `TVS.assertOwner` offers a functionality that is very similar to `_checkOwner` implemented in `OwnableUpgradeable`:

```JavaScript
File: TVS.sol
198:     function _assertOwner() internal view {
199:         if (msg.sender != owner()) {
200:             revert OwnableUnauthorizedAccount(msg.sender);
201:         }
202:     }
---
File: OwnableUpgradeable.sol
81:     function _checkOwner() internal view virtual {
82:         if (owner() != _msgSender()) {
83:             revert OwnableUnauthorizedAccount(_msgSender());
84:         }
85:     }
```

Recommendation: Consider removing `TVS.assertOwner` and using `OwnableUpgradeable.checkOwner` instead.

Customer's response: Fixed with 7447e39

Fix Review: Fix confirmed

I-03 Missing check for deposit amount % 1 gwei == 0

Description: The `BatchDeposit` contract allows to deposit arbitrary amounts; this should not be allowed in general because the deposit contract only allows for exact mutiples of `1 gwei`.

Recommendation: Consider validating amounts in consistency with other fields

Customer's response: Acknowledged: since the check in the beacon deposit contract would revert the transaction if the amount is incorrect, we are OK with not implementing this check in our contract.

I-04 Inconsistent msg.value check in TVS.consolidate vs withdraw

Description: The functions `TVS.withdraw` and `TVS.consolidate` check for the provided `msg.value` in a different way:

- `withdraw` checks that `msg.value` is at least the fees theoretically achievable `(maxFeePerWithdrawal * pubkeys.length)`
- `consolidate` checks that `msg.value` covers at least the fees actually paid, after the fact

Recommendation: While both options make sense, we recommend harmonizing the solutions, and because both the `maxFee` field and `msg.value` are used as limit, we recommend dropping the `maxFee` field, and using `msg.value / num_of_operations` as hard limit for the fee. Alternatively, we recommend enforcing that these two are equal (as done in withdraw), because it's contradictory for a user to accept a higher `maxFee` without providing the value to cover it.

Customer's response: Acknowledged - if we choose to make the checks consistent across both consolidate and withdraw functions, we would have to either redesign the functions completely or the gas cost of these functions would increase significantly due to the extra loop. Considering the `msg.value` checks are still happening in both functions, we decided to optimize for lower gas cost over code consistency.

I-05 Unnecessary loop in AllocationConfig.setBins L81

Description: If we analyze the following loop in `AllocationConfig`:

```
JavaScript
File: AllocationConfig.sol
80:             // Clear extra bins and their nested arrays
81:             for (uint256 i = newLength; i < existingLength;) {
```

```
82:                 r.value.bins[i].nodeOperatorWeightings.pop();
83:                 unchecked {
84:                     ++i;
85:                 }
86:             }
```

One may want to clear the storage of to-be-removed bins for defense-in-depth, or keep it as is to save gas. This code does neither, as it only pops one element that is not guaranteed to be the last.

Recommendation: Consider removing the loop at L81-86 to save gas.

Customer's response: Fixed with 5fc3d1a

Fix Review: Fix confirmed

I-06 Inconsistent initialization between TVSUpgradeable and TVSClone

Description: The `TVSUpgradeable` and `TVSClone` contracts both have an initialize function. However, that of `TVSUpgradeable` has an `initializer` modifier, while that of `TVSClone` doesn't.

Recommendation: We recommend harmonizing the two implementations; since the `_setupSecurity` function that both call already has an `initializer` modifier, having the same modifier also on the outer function is not necessary.

Customer's response: Fixed with deef66b

Fix Review: Fix confirmed

I-07 Missing nonReentrant modifier in TVS.sweep

Description: The `TVS.sweepToBeneficiaryContract` function has a `nonReentrant` modifier; however, the `TVS.sweep` function doesn't, while implicitly offering the same reentrancy opportunity through the ETH transfer.

Recommendation: We recommend harmonizing the two functions, in particular adding `nonReentrant` to `TVS.sweep` out of greater precaution.

Customer's response: Fixed with 8b7bed6

Fix Review: Fix confirmed

I-08 Implementation contracts don't disable initializers

Description: The contracts `OrchestratorUpgradeableBase`, `OrchestratorFactory`, `TVSUpgradeable` are all `Initializable` children, meant to be used behind a proxy. However, in their constructors, they don't call the `_disableInitializers()` function as recommended by the Initializable contract.

Recommendation: Consider adding `_disableInitializers()` calls in the constructors of these contracts.

Customer's response: Fixed with 8681414 and 3ccaf2c

Fix Review: Fix confirmed

I-09 Error-prone TVSFlexibleImmutable.executeBatch payable

Description: The `TVSFlexibleImmutalbe.executeBatch` function allows making multiple `delegatecall` calls within itself, which is a `payable` function. This can be problematic because if two or more of the called functions make use of `msg.value`, their logic will most likely misbehave.

Also, when non-zero value is provided to `executeBatch`, delegatecall to non-`payable` functions will most likely revert.

Recommendation: Consider documenting clearly the limitations:
- when `msg.value` is passed, only one `delegatecall` should be made
- when `msg.value` is passed, any `delegatecall` to non-`payable` functions will fail

Customer's response: Fixed with 46a1e7d

Fix Review: Fix confirmed

I-10 OrchestratorBase grantRole, revokeRole, renounceRole check caller role twice

Description: If we look at the code of these functions, we see that they have an onlyRole modifier, but also call the corresponding public function of the parent contract, which also has an onlyRole modifier:

```
JavaScript
File: OrchestratorBase.sol
483:     /// @inheritdoc AccessControlUpgradeable
484:     function grantRole(bytes32 role, address account) public
override onlyRole(DEFAULT_ADMIN_ROLE) isNotPaused {
485:         if (role == DEFAULT_ADMIN_ROLE) {
486:             revert InvalidOperation();
487:         }
488:         super.grantRole(role, account);
489:     }
```

Recommendation: We recommend instead calling `super._grantRole`, `super._revokeRole`, `super._renounceRole` to avoid the redundant check and save gas.

Customer's response: Fixed with 5a9cfef

Fix Review: Fix confirmed

I-11 Duplicate events emitted by BatchSweep::_sweepToBeneficiary and TVS:_sweep

Description: Both these functions emit a `Swept` event, with similar arguments.

Recommendation: We recommend removing the `BatchSweep.Swept` event, as the event emitted by the TVS is more accurate with updated values for the destination address and sweep amount.

Customer's response: Fixed with 4f9b883

Fix Review: Fix confirmed

I-12 Orchestrators cannot create immutable TVS instances

Description: Orchestrator contracts offer a factory method, `createValidatorSets`, that allows creation of TVS instances in batch, given an ERC-1967 implementation beacon.

This factory method, however, is not compatible with any of the ways immutable TVS instances can be created, including the ERC-1967-compatible `TVSClone` contract, because it is hardcoded to call an initializer with three `address` arguments:

```
JavaScript
File: BatchUpgradeableTVSCreator.sol
41:     new TVSBeaconProxy(
42:         _beacon,
```

```
43:         abi.encodeWithSignature("initialize(address,address,address)",...)
44:     )
```

Recommendations: We recommend adding a third, unused argument to the `TVSClone initialize` function, so the `createValidatorSets` factory can offer the option to create immutable TVS instances when needed.

Customer's response: Acknowledged - we have updated the spec. This is out of scope.

I-13 `sweepToBeneficiaryContract` isn't callable through the Orchestrator

Description: There are 2 functions that execute `_sweep` in TVS, `sweep` & `sweepToBeneficiaryContract`. Currently only `sweep` is available to be called through the Orchestrator, `sweepToBeneficiaryContract` cannot be called, because there is no function that uses it.

Recommendations: Add a function to `BatchSweep` that calls `sweepToBeneficiaryContract`

Customer's response: Acknowledged - we have updated the spec. This is out of scope.

I-14 Orchestrator misses a `receive payable` function

Description: The `OrchestratorBase` contract, and consequently all its child contracts, miss a `receive` or `fallback payable` function.

Recommendations: Consider adding a receive function to the `OrchestratorBase` contract, and having Orchestrator as beneficiary seems a sensible choice.

Customer's response: Acknowledged - this is a design choice we have made since the orchestrator is not expected to be a beneficiary, and therefore is not expected to receive ETH from sweeps. Not having the `receive payable` function also prevents any unsolicited ETH from entering the system. That's why we think that this is not a bug.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.